

Network Management

Although network management is implemented at the application layer of the TCP/IP protocol suite, we have postponed the discussion of this issue until now to be able to discuss it in more detail. Network management plays an important role in the Internet as it becomes larger and larger. The failure of a single device may interrupt the communication from one point of the Internet to the other. In this chapter, we first discuss the areas of network management. We then discuss how one of these areas is implemented at the application layer of the TCP/IP suite. We have divided this chapter into three sections:

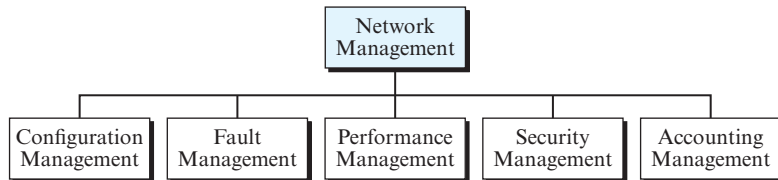
- ❑ In the first section, we introduce the concept of network management and discuss five general areas of network management: configuration, fault, performance, security, and accounting. *Configuration management* is related to the status of each entity and its relationship to other entities. *Fault management* is the area of network management that handles issues related to interruptions in the system. *Performance management* tries to monitor and control the network to ensure that it is running as efficiently as possible. *Security management* is responsible for controlling access to the network based on predefined policy. *Accounting management* is the controlling of users' access to network resources through charges.
- ❑ In the second section, we discuss Simple Network Management Protocol (SNMP) as a framework for managing devices in an internet using the TCP/IP protocol suite and show how a manager as a host runs an SNMP client and any agents as a router or host runs a server program. We define the three components of the management protocol in the Internet. We also define Structure of Management Information (SMI) as the language that specifies how data types and objects in SNMP should be identified. We then introduce Management Information Base (MIB), which designates the objects to be managed in SNMP according to the rules defined in SMI.
- ❑ In the third section, we give a brief discussion of a standard that provides the methods and rules to define data and objects. This section is very brief and only introduces the subject. Part of it is used by SMI in the second section.

9.1 INTRODUCTION

We can define *network management* as monitoring, testing, configuring, and troubleshooting network components to meet a set of requirements defined by an organization. These requirements include the smooth, efficient operation of the network that provides the predefined quality of service for users. To accomplish this task, a network management system uses hardware, software, and humans.

The International Organization for Standardization (ISO) defines five areas of network management: configuration management, fault management, performance management, security management, and accounting management, as shown in Figure 9.1.

Figure 9.1 Areas of network management



Although some organizations include other areas, such as cost management, we believe the ISO taxonomy is specific to network management. For example, cost management is a general management area for any management system and not just for network management.

9.1.1 Configuration Management

A large network is usually made up of hundreds of entities that are physically or logically connected to each other. These entities have an initial configuration when the network is set up, but can change with time. Desktop computers may be replaced by others; application software may be updated to a newer version; and users may move from one group to another. The *configuration management* system must know, at any time, the status of each entity and its relation to other entities. Configuration management can be divided into two subsystems: *reconfiguration* and *documentation*.

Reconfiguration

Reconfiguration can be a daily occurrence in a large network. There are three types of reconfiguration: *hardware reconfiguration*, *software reconfiguration*, and *user-account reconfiguration*.

Hardware Reconfiguration

Hardware reconfiguration covers all changes to the hardware. For example, a desktop computer may need to be replaced. A router may need to be moved to another part of the network. A subnetwork may be added or removed from the network. All of these need the time and attention of network management. In a large network, there must be specialized personnel trained for quick and efficient hardware reconfiguration. Unfortunately, this type of reconfiguration cannot be automated and must be manually handled case by case.

Software Reconfiguration

Software reconfiguration covers all changes to the software. For example, new software may need to be installed on servers or clients. An operating system may need updating. Fortunately, most software reconfiguration can be automated. For example, an update for an application on some or all clients can be electronically downloaded from the server.

User-Account Reconfiguration

User-account reconfiguration is not simply adding or deleting users on a system. We must also consider the user privileges, both as an individual and as a member of a group. For example, a user may have both read and write permission with regard to some files, but only read permission with regard to other files. User-account reconfiguration can be, to some extent, automated. For example, in a college or university, at the beginning of each quarter or semester, new students are added to the system. The students are normally grouped according to the courses they take or the majors they pursue. The members of each group have specific privileges; computer science students may need to access a server providing different computer language facilities, while engineering students may need to access servers that provide computer assisted design (CAD) software.

Documentation

The original network configuration and each subsequent change must be recorded meticulously. This means that there must be documentation for hardware, software, and user accounts.

Hardware Documentation

Hardware documentation normally involves two sets of documents: maps and specifications.

Maps *Maps* track each piece of hardware and its connection to the network. There can be one general map that shows the logical relationships between subnetworks. There can also be a second general map that shows the physical location of each subnetwork. For each subnetwork, then, there is one or more maps that show all pieces of equipment. The maps use some kind of standardization to be easily read and understood by current and future personnel.

Specifications Maps are not enough per se. Each piece of hardware also needs to be documented. There must be a set of *specifications* for each piece of hardware connected to the network. These specifications must include information such as hardware type, serial number, vendor (address and phone number), time of purchase, and warranty information.

Software Documentation

All software must also be documented. *Software documentation* includes information such as the software type, the version, the time installed, and the license agreement.

User-Account Documentation

Most operating systems have a utility that allows *user account documentation*. The management must make sure that the files with this information are updated and secured. Some operating systems record access privileges in two documents; one shows

all files and access types for each user; the other shows the list of users that have access to a particular file.

9.1.2 Fault Management

Complex networks today are made up of hundreds and sometimes thousands of components. Proper operation of the network depends on the proper operation of each component individually and in relation to each other. *Fault management* is the area of network management that handles this issue. An effective fault management system has two subsystems: reactive fault management and proactive fault management.

Reactive Fault Management

A *reactive fault management* system is responsible for detecting, isolating, correcting, and recording faults. It handles short-term solutions to faults.

Detecting Fault

The first step taken by a reactive fault management system is to find the exact location of the fault. A fault is defined as an abnormal condition in the system. When a fault occurs, either the system stops working properly or the system creates excessive errors. A good example of a fault is a damaged communication medium.

Isolating Fault

The next step taken by a reactive fault management system is isolating the fault. A fault, if isolated, usually affects only a few users. After isolation, the affected users are immediately notified and given an estimated time of correction.

Correcting Fault

The next step is correcting the fault. This may involve replacing or repairing the faulty components.

Recording Fault

After the fault is corrected, it must be documented. The record should show the exact location of the fault, the possible cause, the action or actions taken to correct the fault, the cost, and the time it took for each step. Documentation is extremely important for several reasons:

- ❑ The problem may reoccur. Documentation can help the present or future administrator or technician solve a similar problem.
- ❑ The frequency of the same kind of failure is an indication of a major problem in the system. If a fault happens frequently in one component, the component should be replaced with a similar one or the whole system should be changed to avoid the use of that type of component.
- ❑ The statistic is helpful to another part of network management, performance management.

Proactive Fault Management

Proactive fault management tries to prevent faults from occurring. Although this is not always possible, some types of failures can be predicted and prevented. For example, if a manufacturer specifies a lifetime for a component or a part of a component, it is a good strategy to replace it before that time. As another example, if a fault happens

frequently at one particular point of a network, it is wise to carefully reconfigure the network to prevent the fault from happening again.

9.1.3 Performance Management

Performance management, which is closely related to fault management, tries to monitor and control the network to ensure that it is running as efficiently as possible. Performance management tries to quantify performance using some measurable quantity, such as capacity, traffic, throughput, or response time. Some protocols, such as SNMP, which is discussed in this chapter, can be used in performance management.

Capacity

One factor that must be monitored by a performance management system is the *capacity* of the network. Every network has a limited capacity and the performance management system must ensure that it is not used above this capacity. For example, if a LAN is designed for 100 stations at an average data rate of 2 Mbps, it will not operate properly if 200 stations are connected to the network. The data rate will decrease and blocking may occur.

Traffic

Traffic can be measured in two ways: internally and externally. Internal traffic is measured by the number of packets (or bytes) travelling inside the network. External traffic is measured by the exchange of packets (or bytes) outside the network. During peak hours, when the system is heavily used, blocking may occur if there is excessive traffic.

Throughput

We can measure the *throughput* of an individual device (such as a router) or a part of the network. Performance management monitors the throughput to make sure that it is not reduced to unacceptable levels.

Response Time

Response time is normally measured from the time a user requests a service to the time the service is granted. Other factors such as capacity and traffic can affect the response time. Performance management monitors the average response time and the peak-hour response time. Any increase in response time is a very serious condition as it is an indication that the network is working above its capacity.

9.1.4 Security Management

Security management is responsible for controlling access to the network based on predefined policy. In Chapter 10 we will discuss security tools such as encryption and authentication. Encryption allows privacy for users; authentication forces the users to identify themselves.

9.1.5 Accounting Management

Accounting management is the controlling of users' access to network resources through charges. Under accounting management, individual users, departments, divisions, or even projects are charged for the services they receive from the network. Charging does not necessarily mean cash transfer; it may mean debiting the departments or divisions for

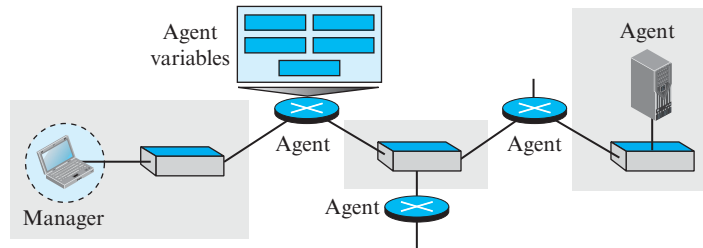
budgeting purposes. Today, organizations use an accounting management system for the following reasons:

- ❑ It prevents users from monopolizing limited network resources.
- ❑ It prevents users from using the system inefficiently.
- ❑ Network managers can do short- and long-term planning based on the demand for network use.

9.2 SNMP

Several network management standards have been devised during the last few decades. The most important one is **Simple Network Management Protocol (SNMP)**, used by the Internet. We discuss this standard in this section. SNMP is a framework for managing devices in an internet using the TCP/IP protocol suite. It provides a set of fundamental operations for monitoring and maintaining an internet. SNMP uses the concept of manager and agent. That is, a manager, usually a host, controls and monitors a set of agents, usually routers or servers (see Figure 9.2).

Figure 9.2 SNMP concept



SNMP is an application-level protocol in which a few manager stations control a set of agents. The protocol is designed at the application level so that it can monitor devices made by different manufacturers and installed on different physical networks. In other words, SNMP frees management tasks from both the physical characteristics of the managed devices and the underlying networking technology. It can be used in a heterogeneous internet made of different LANs and WANs connected by routers made by different manufacturers.

9.2.1 Managers and Agents

A management station, called a *manager*, is a host that runs the SNMP client program. A managed station, called an *agent*, is a router (or a host) that runs the SNMP server program. Management is achieved through simple interaction between a manager and an agent.

The agent keeps performance information in a database. The manager has access to the values in the database. For example, a router can store in appropriate variables the number of packets received and forwarded. The manager can fetch and compare the values of these two variables to see if the router is congested or not.

The manager can also make the router perform certain actions. For example, a router periodically checks the value of a reboot counter to see when it should reboot

itself. It reboots itself, for example, if the value of the counter is 0. The manager can use this feature to reboot the agent remotely at any time. It simply sends a packet to force a 0 value in the counter.

Agents can also contribute to the management process. The server program running on the agent can check the environment and, if it notices something unusual, it can send a warning message (called a *trap*) to the manager.

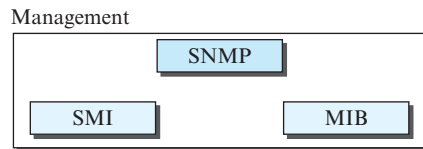
In other words, management with SNMP is based on three basic ideas:

1. A manager checks an agent by requesting information that reflects the behavior of the agent.
2. A manager forces an agent to perform a task by resetting values in the agent database.
3. An agent contributes to the management process by warning the manager of an unusual situation.

9.2.2 Management Components

To do management tasks, SNMP uses two other protocols: **Structure of Management Information (SMI)** and **Management Information Base (MIB)**. In other words, management on the Internet is done through the cooperation of three protocols: SNMP, SMI, and MIB, as shown in Figure 9.3.

Figure 9.3 Components of network management on the Internet



Let us elaborate on the interactions between these protocols.

Role of SNMP

SNMP has some very specific roles in network management. It defines the format of the packet to be sent from a manager to an agent and vice versa. It also interprets the result and creates statistics (often with the help of other management software). The packets exchanged contain the object (variable) names and their status (values). SNMP is responsible for reading and changing these values.

SNMP defines the format of packets exchanged between a manager and an agent. It reads and changes the status of objects (values of variables) in SNMP packets.

Role of SMI

To use SNMP, we need rules for naming objects. This is particularly important because the objects in SNMP form a hierarchical structure (an object may have a parent object and some child objects). Part of a name can be inherited from the parent. We also need rules to define the types of objects. What types of objects are handled by SNMP? Can SNMP handle simple types or structured types? How many simple types are available?

What are the sizes of these types? What is the range of these types? In addition, how are each of these types encoded?

SMI defines the general rules for naming objects, defining object types (including range and length), and showing how to encode objects and values.

We need these universal rules because we do not know the architecture of the computers that send, receive, or store these values. The sender may be a powerful computer in which an integer is stored as 8-byte data; the receiver may be a small computer that stores an integer as 4-byte data.

SMI is a protocol that defines these rules. However, we must understand that SMI only defines the rules; it does not define how many objects are managed in an entity or which object uses which type. SMI is a collection of general rules to name objects and to list their types. The association of an object with the type is not done by SMI.

Role of MIB

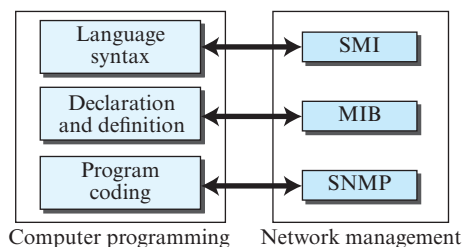
We hope it is clear that we need another protocol. For each entity to be managed, this protocol must define the number of objects, name them according to the rules defined by SMI, and associate a type to each named object. This protocol is MIB. MIB creates a set of objects defined for each entity in a manner similar to that of a database (mostly metadata in a database, names and types without values).

MIB creates a collection of named objects, their types, and their relationships to each other in an entity to be managed.

An Analogy

Before discussing each of these protocols in more detail, let us give an analogy. The three network management components are similar to what we need when we write a program in a computer language to solve a problem. Figure 9.4 shows the analogy.

Figure 9.4 Comparing computer programming and network management



Syntax: SMI

Before we write a program, the syntax of the language (such as C or Java) must be pre-defined. The language also defines the structure of variables (simple, structured, pointer, and so on) and how the variables must be named. For example, a variable name must be 1 to n characters in length and start with a letter followed by alphanumeric characters. The language also defines the type of data to be used (integer, real, character, etc.).

In programming the rules are defined by the syntax of the language. In network management the rules are defined by SMI.

Object Declaration and Definition: MIB

Most computer languages require that objects be declared and defined in each specific program. Declaration and definition creates objects using predefined types and allocates memory location for them. For example, if a program has two variables (an integer named *counter* and an array named *grades* of type *char*), they must be declared at the beginning of the program:

```
int counter;
char grades [40];
```

MIB does this task in network management. MIB names each object and defines the type of the objects. Because the type is defined by SMI, SNMP knows the range and size.

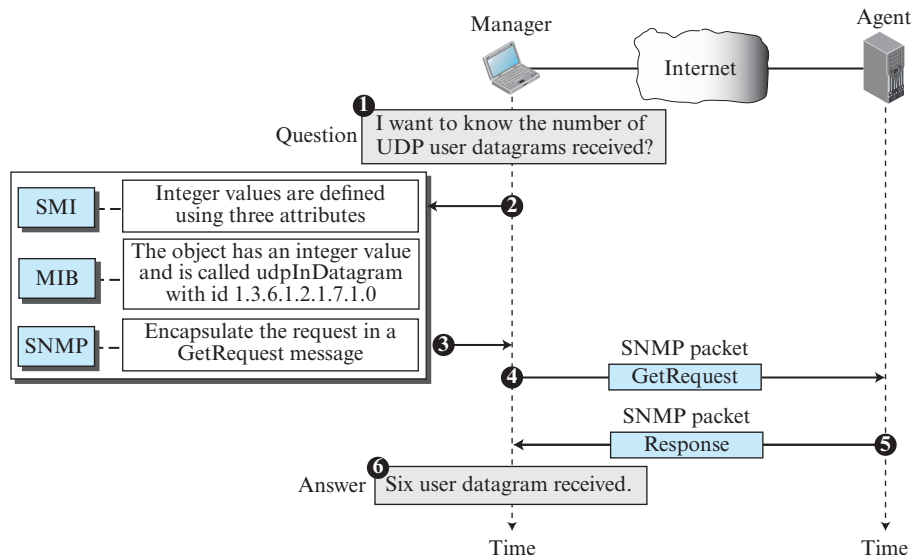
Program Coding: SNMP

After declaration in programming, the program needs to write statements to store values in the variables and change them if needed. SNMP does this task in network management. SNMP stores, changes, and interprets the values of objects already declared by MIB according to the rules defined by SMI.

9.2.3 An Overview

Before discussing each component in more detail, let us show how each of these components is involved in a simple scenario. This is an overview that will be developed later, at the end of the chapter. A manager station (SNMP client) wants to send a message to an agent station (SNMP server) to find the number of UDP user datagrams received by the agent. Figure 9.5 shows an overview of steps involved.

Figure 9.5 Management overview



MIB is responsible for finding the object that holds the number of UDP user datagrams received. SMI, with the help of another embedded protocol, is responsible for encoding the name of the object. SNMP is responsible for creating a message, called a GetRequest message, and encapsulating the encoded message. Of course, things are more complicated than this simple overview, but we first need more details of each protocol.

9.2.4 SMI

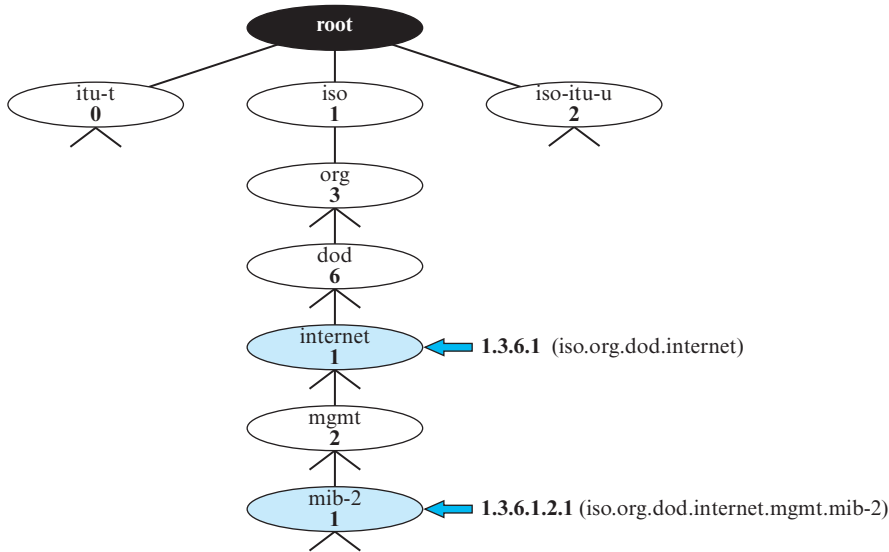
The Structure of Management Information, version 2 (SMIv2) is a component for network management. SMI is a guideline for SNMP. It emphasizes three attributes to handle an object: name, data type, and encoding method. Its functions are:

- ❑ To name objects.
- ❑ To define the type of data that can be stored in an object.
- ❑ To show how to encode data for transmission over the network.

Name

SMI requires that each managed object (such as a router, a variable in a router, a value, etc.) have a unique name. To name objects globally, SMI uses an **object identifier**, which is a hierarchical identifier based on a tree structure (see Figure 9.6).

Figure 9.6 Object identifier in SMI



The tree structure starts with an unnamed root. Each object can be defined using a sequence of integers separated by dots. The tree structure can also define an object using a sequence of textual names separated by dots.

The integer-dot representation is used in SNMP. The name-dot notation is used by people. For example, the following shows the same object in two different notations.

iso.org.dod.internet.mgmt.mib-2	↔	1.3.6.1.2.1
---------------------------------	---	-------------

The objects that are used in SNMP are located under the *mib-2* object, so their identifiers always start with 1.3.6.1.2.1.

Type

The second attribute of an object is the type of data stored in it. To define the data type, SMI uses **Abstract Syntax Notation One (ASN.1)** definitions and adds some new definitions. In other words, SMI is both a subset and a superset of ASN.1. (We discuss ASN.1 at the last section of this chapter).

SMI has two broad categories of data type: *simple* and *structured*. We first define the simple types and then show how the structured types can be constructed from the simple ones.

Simple Type

The **simple data types** are atomic data types. Some of them are taken directly from ASN.1; some are added by SMI. The most important ones are given in Table 9.1. The first five are from ASN.1; the next seven are defined by SMI.

Table 9.1 Data types

Type	Size	Description
INTEGER	4 bytes	An integer with a value between -2^{31} and $2^{31}-1$
Integer32	4 bytes	Same as INTEGER
Unsigned32	4 bytes	Unsigned with a value between 0 and $2^{32}-1$
OCTET STRING	Variable	Byte-string up to 65,535 bytes long
OBJECT IDENTIFIER	Variable	An object identifier
IPAddress	4 bytes	An IP address made of four integers
Counter32	4 bytes	An integer whose value can be incremented from zero to 2^{32} ; when it reaches its maximum value it wraps back to zero
Counter64	8 bytes	64-bit counter
Gauge32	4 bytes	Same as Counter32, but when it reaches its maximum value, it does not wrap; it remains there until it is reset
TimeTicks	4 bytes	A counting value that records time in 1/100ths of a second
BITS		A string of bits
Opaque	Variable	Uninterpreted string

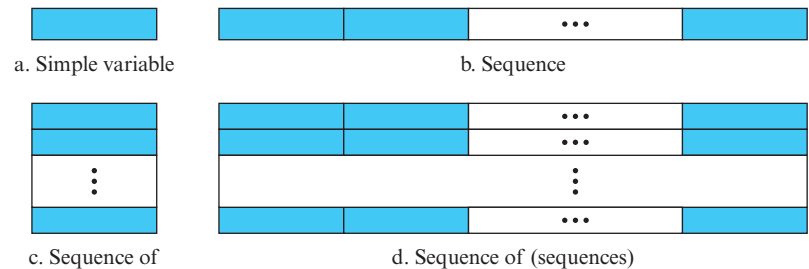
Structured Type

By combining simple and structured data types, we can make new structured data types. SMI defines two **structured data types**: *sequence* and *sequence of*.

- **Sequence.** A *sequence* data type is a combination of simple data types, not necessarily of the same type. It is analogous to the concept of a *struct* or a *record* used in programming languages such as C.
- **Sequence of.** A *sequence of* data type is a combination of simple data types all of the same type or a combination of sequence data types all of the same type. It is analogous to the concept of an *array* used in programming languages such as C.

Figure 9.7 shows a conceptual view of data types.

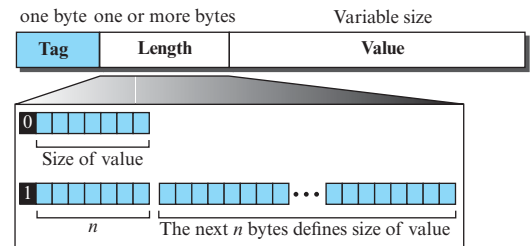
Figure 9.7 Conceptual data types



Encoding Method

SMI uses another standard, **Basic Encoding Rules (BER)**, to encode data to be transmitted over the network. BER specifies that each piece of data be encoded in triplet format: tag, length, and value (TLV), as illustrated in Figure 9.8.

Figure 9.8 Encoding format



The tag is a 1-byte field that defines the type of data. Table 9.2 shows the data types we use in this chapter and their tags in hexadecimal numbers. The length field is 1 or more bytes. If it is one byte, the most significant bit must be 0. The other 7 bits define the length of the data. If it is more than one byte, the most significant bit of the first byte must be 1. The other 7 bits of the first byte specify the number of bytes needed to define the length. The value field codes the value of the data according to the rules defined in BER.

Table 9.2 Codes for data types

Data Type	Tag (Hex)	Data Type	Tag (Hex)
INTEGER	02	IPAddress	40
OCTET STRING	04	Counter	41
OBJECT IDENTIFIER	06	Gauge	42
NULL	05	TimeTicks	43
SEQUENCE, SEQUENCE OF	30	Opaque	44

Example 9.1

Figure 9.9 shows how to define INTEGER 14. The size of the length field is from Table 9.1.

Figure 9.9 Example 9.1: INTEGER 14

0x02	0x04	0x00	0x00	0x00	0x0E
Tag (integer)	Length (4 bytes)	Value (14)			

Example 9.2

Figure 9.10 shows how to define the OCTET STRING “HI.”

Figure 9.10 Example 9.2: OCTET STRING “HI”

0x04	0x02	0x48	0x49
Tag (String)	Length (2 bytes)	Value (H)	Value (I)

Example 9.3

Figure 9.11 shows how to define ObjectIdentifier 1.3.6.1 (iso.org.dod.internet).

Figure 9.11 Example 9.3: ObjectIdentifier 1.3.6.1

0x06	0x04	0x01	0x03	0x06	0x01
Tag (ObjectId)	Length (4 bytes)	Value (1)	Value (3)	Value (6)	Value (1)
←1.3.6.1 (iso.org.dod.internet)→					

Example 9.4

Figure 9.12 shows how to define IPAddress 131.21.14.8.

Figure 9.12 Example 9.4: IPAddress 131.21.14.8

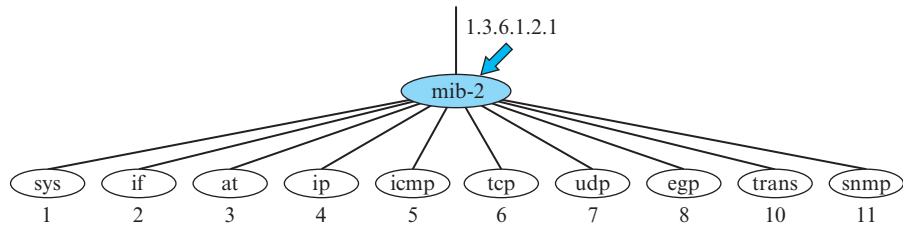
0x40	0x04	0x83	0x15	0x0E	0x08
Tag (IPAddress)	Length (4 bytes)	Value (131)	Value (21)	Value (14)	Value (8)
←131.21.14.8→					

9.2.5 MIB

The Management Information Base, version 2 (MIB2) is the second component used in network management. Each agent has its own MIB2, which is a collection of all the objects that the manager can manage. (See Figure 9.13.)

The objects in MIB2 are categorized under several groups: system, interface, address translation, ip, icmp, tcp, udp, egp, transmission, and snmp (note that group 9

Figure 9.13 *Some mib-2 groups*



is deprecated). These groups are under the mib-2 object in the object identifier tree. Each group has defined variables and/or tables.

The following is a brief description of some of the objects:

- ❑ **sys** This object (*system*) defines general information about the node (system), such as the name, location, and lifetime.
- ❑ **if** This object (*interface*) defines information about all of the interfaces of the node including interface number, physical address, and IP address.
- ❑ **at** This object (*address translation*) defines the information about the ARP table.
- ❑ **ip** This object defines information related to IP, such as the routing table and the IP address.
- ❑ **icmp** This object defines information related to ICMP, such as the number of packets sent and received and total errors created.
- ❑ **tcp** This object defines general information related to TCP, such as the connection table, time-out value, number of ports, and number of packets sent and received.
- ❑ **udp** This object defines general information related to UDP, such as the number of ports and number of packets sent and received.
- ❑ **egp** Objects related to the operation of EGP.
- ❑ **trans** Objects related to the specific method of transmission (future use)
- ❑ **snmp** This object defines general information related to SNMP itself.

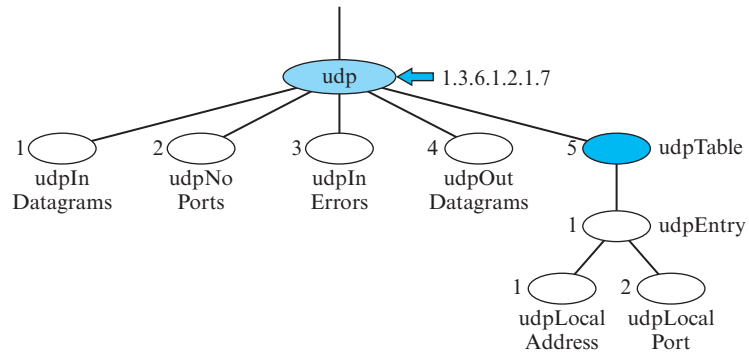
Accessing MIB Variables

To show how to access different variables, we use the udp group as an example. There are four simple variables in the udp group and one sequence of (table of) records. Figure 9.14 shows the variables and the table. We will show how to access each entity.

Simple Variables

To access any of the simple variables, we use the id of the group (1.3.6.1.2.1.7) followed by the id of the variable. The following shows how to access each variable.

udpInDatagrams	→	1.3.6.1.2.1.7.1
udpNoPorts	→	1.3.6.1.2.1.7.2
udpInErrors	→	1.3.6.1.2.1.7.3
udpOutDatagrams	→	1.3.6.1.2.1.7.4

Figure 9.14 *udp group*

However, these object identifiers define the variable, not the instance (contents). To show the instance, or the contents, of each variable, we must add an instance suffix. The instance suffix for a simple variable is simply a zero. In other words, to show an instance of the above variables, we use the following:

udpInDatagrams.0	→	1.3.6.1.2.1.7.1.0
udpNoPorts.0	→	1.3.6.1.2.1.7.2.0
udpInErrors.0	→	1.3.6.1.2.1.7.3.0
udpOutDatagrams.0	→	1.3.6.1.2.1.7.4.0

Tables

To identify a table, we first use the table id. The udp group has only one table (with id 5), as illustrated in Figure 9.15. So to access the table, we use the following:

udpTable → **1.3.6.1.2.1.7.5**

However, the table is not at the leaf level in the tree structure. We cannot access the table; we define the entry (sequence) in the table (with id of 1), as follows:

udpEntry → **1.3.6.1.2.1.7.5.1**

This entry is also not a leaf and we cannot access it. We need to define each entity (field) in the entry.

udpLocalAddress → **1.3.6.1.2.1.7.5.1.1**
udpLocalPort → **1.3.6.1.2.1.7.5.1.2**

These two variables are at the leaf level of the tree. Although we can access their instances, we need to define *which* instance. At any moment, the table can have several values for each local address/local port pair. To access a specific instance (row) of the table, we add the index to the above ids. In MIB, the indexes of arrays are not integers (unlike most programming languages). The indexes are based on the value of one or more fields in the entries. In our example, the udpTable is indexed based on both the local address and the local port number. For example, Figure 9.16 shows a table with four rows

Figure 9.15 *udp variables and tables*

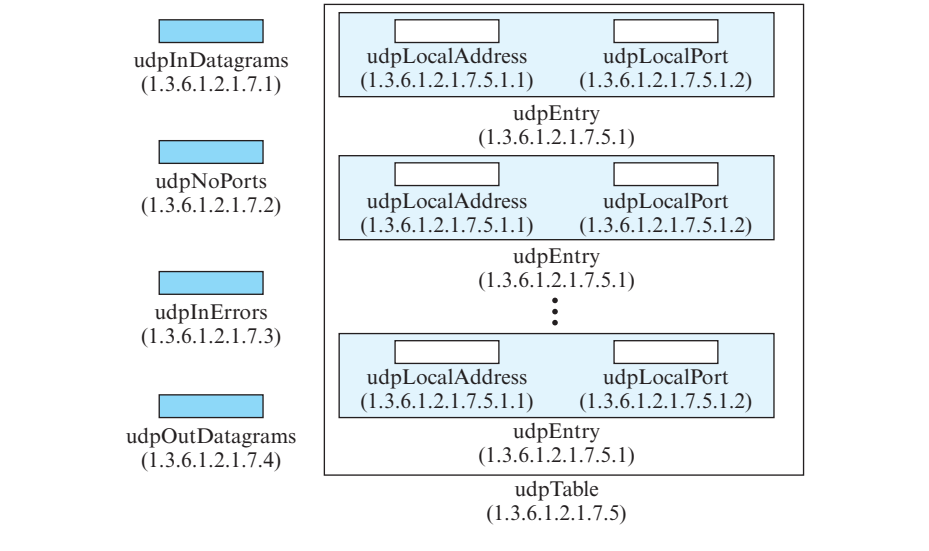


Figure 9.16 *Indexes for udpTable*

181.23.45.14	23
1.3.6.1.2.1.7.5.1.1.181.23.45.14.23	1.3.6.1.2.1.7.5.1.2.181.23.45.14.23
192.13.5.10	161
1.3.6.1.2.1.7.5.1.1.192.13.5.10.161	1.3.6.1.2.1.7.5.1.2.192.13.5.10.161
227.2.45.18	180
1.3.6.1.2.1.7.5.1.1.227.2.45.18.180	1.3.6.1.2.1.7.5.1.2.227.2.45.18.180
230.20.5.24	212
1.3.6.1.2.1.7.5.1.1.230.20.5.24.212	1.3.6.1.2.1.7.5.1.2.230.20.5.24.212

and values for each field. The index of each row is a combination of two values. To access the instance of the local address for the first row, we use the identifier augmented with the instance index:

udpLocalAddress.181.23.45.14.23 → **1.3.6.1.2.7.5.1.1.181.23.45.14.23**

9.2.6 SNMP

SNMP uses both SMI and MIB in Internet network management. It is an application program that allows:

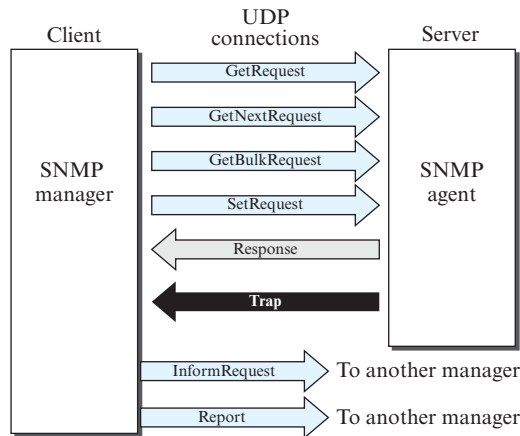
- ❑ A manager to retrieve the value of an object defined in an agent.

- ❑ A manager to store a value in an object defined in an agent.
- ❑ An agent to send an alarm message about an abnormal situation to the manager.

PDUs

SNMPv3 defines eight types of protocol data units (or PDUs): *GetRequest*, *GetNextRequest*, *GetBulkRequest*, *SetRequest*, *Response*, *Trap*, *InformRequest*, and *Report* (see Figure 9.17).

Figure 9.17 SNMP PDUs



GetRequest

The *GetRequest* PDU is sent from the manager (client) to the agent (server) to retrieve the value of a variable or a set of variables.

GetNextRequest

The *GetNextRequest* PDU is sent from the manager to the agent to retrieve the value of a variable. The retrieved value is the value of the object following the defined *ObjectId* in the PDU. It is mostly used to retrieve the values of the entries in a table. If the manager does not know the indexes of the entries, it cannot retrieve the values. However, it can use *GetNextRequest* and define the *ObjectId* of the table. Because the first entry has the *ObjectId* immediately after the *ObjectId* of the table, the value of the first entry is returned. The manager can use this *ObjectId* to get the value of the next one, and so on.

GetBulkRequest

The *GetBulkRequest* PDU is sent from the manager to the agent to retrieve a large amount of data. It can be used instead of multiple *GetRequest* and *GetNextRequest* PDUs.

SetRequest

The *SetRequest* PDU is sent from the manager to the agent to set (store) a value in a variable.

Response

The Response PDU is sent from an agent to a manager in response to GetRequest or GetNextRequest. It contains the value(s) of the variable(s) requested by the manager.

Trap

The **Trap** (also called SNMPv2 Trap to distinguish it from SNMPv1 Trap) PDU is sent from the agent to the manager to report an event. For example, if the agent is rebooted, it informs the manager and reports the time of rebooting.

InformRequest

The InformRequest PDU is sent from one manager to another remote manager to get the value of some variables from agents under the control of the remote manager. The remote manager responds with a Response PDU.

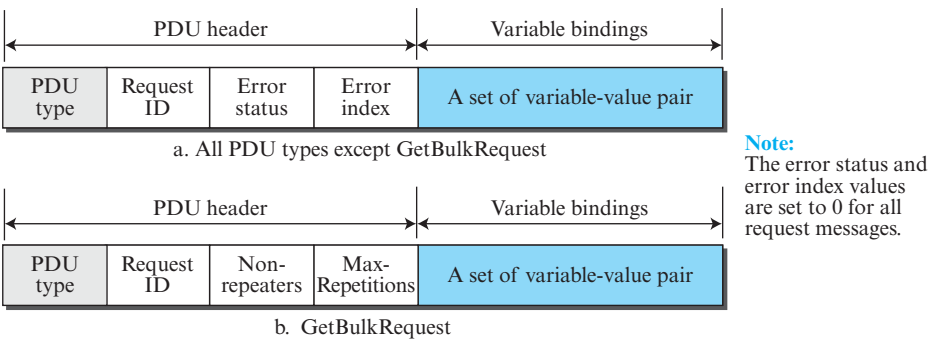
Report

The Report PDU is designed to report some types of errors between managers. It is not yet in use.

Format

The format for the eight SNMP PDUs is shown in Figure 9.18. The GetBulkRequest PDU differs from the others in two areas, as shown in the figure.

Figure 9.18 SNMP PDU format



The fields are listed below:

- ❑ **PDU type.** This field defines the type of the PDU (see Table 9.3) .

Table 9.3 PDU types

Type	Tag (Hex)	Type	Tag (Hex)
GetRequest	A0	GetBulkRequest	A5
GetNextRequest	A1	InformRequest	A6
Response	A2	Trap (SNMPv2)	A7
SetRequest	A3	Report	A8

- ❑ **Request ID.** This field is a sequence number used by the manager in a request PDU and repeated by the agent in a response. It is used to match a request to a response.

- ❑ **Error status.** This is an integer that is used only in response PDUs to show the types of errors reported by the agent. Its value is 0 in request PDUs. Table 9.4 lists the types of errors that can occur.

Table 9.4 Types of errors

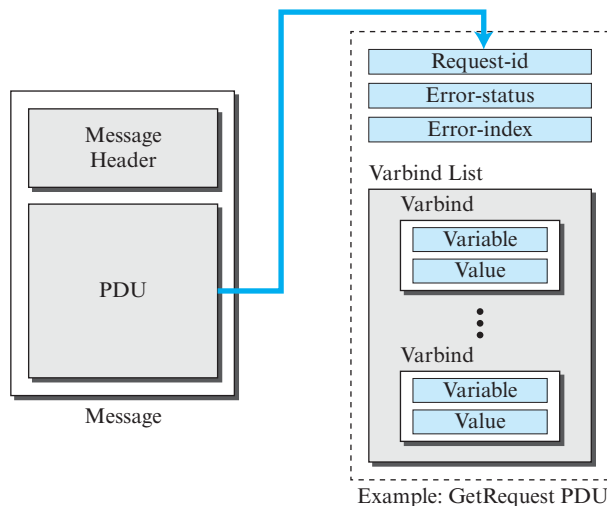
Status	Name	Meaning
0	noError	No error
1	tooBig	Response too big to fit in one message
2	noSuchName	Variable does not exist
3	badValue	The value to be stored is invalid
4	readOnly	The value cannot be modified
5	genErr	Other errors

- ❑ **Non-repeaters.** This field is used only in GetBulkRequest PDU. The field defines the number of non-repeating (regular objects) at the start of the variable-value list.
- ❑ **Error index.** The error index is an offset that tells the manager which variable caused the error.
- ❑ **Max-repetitions.** This field is also used only in GetBulkRequest PDU. The field defines the maximum number of iteration in the table to read all repeating object.
- ❑ **Variable-value pair list.** This is a set of variables with the corresponding values the manager wants to retrieve or set. The values are null in request PDUs.

Messages

SNMP does not send only PDUs, it embeds each PDU in a message. A message is made of a message header followed by the corresponding PDU as shown in Figure 9.19. The format of the message header, which depends on the version and the security provision, is not shown in the figure. We leave the details to some specific text.

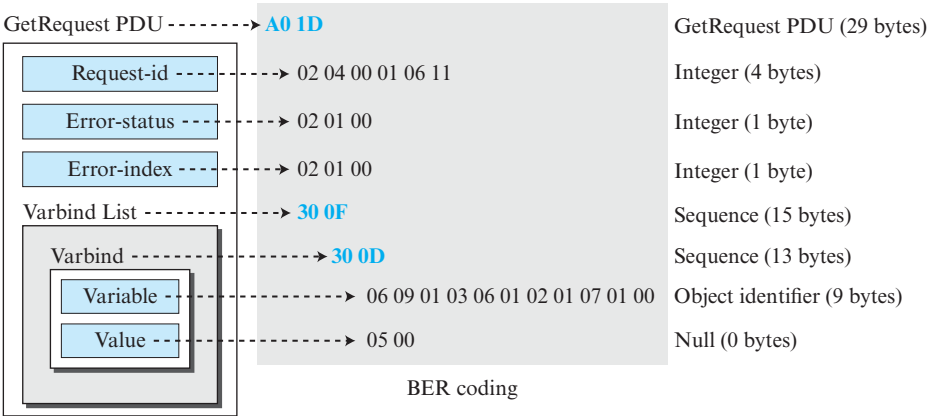
Figure 9.19 SNMP message



Example 9.5

In this example, a manager station (SNMP client) uses a message with a GetRequest PDU to retrieve the number of UDP datagrams that a router has received (Figure 9.20).

Figure 9.20 Example 9.5



There is only one Varbind sequence. The corresponding MIB variable related to this information is `udpInDatagrams` with the object identifier 1.3.6.1.2.1.7.1.0. The manager wants to retrieve a value (not to store a value), so the value defines a null entity. The bytes to be sent are shown in hexadecimal representation.

The Varbind list has only one Varbind. The variable is of type 06 and length 09. The value is of type 05 and length 00. The whole Varbind is a sequence of length 0D (13). The Varbind list is also a sequence of length 0F (15). The GetRequest PDU is of length 1D (29).

Note that we have intended the bytes to show the inclusion of simple data types inside a sequence or the inclusion of sequences and simple data type inside larger sequences. Note that the PDU itself is like a sequence, but its tag is A0 in hexadecimal.

Figure 9.21 shows the actual message sent. We assume that the message header is made of ten bytes. The actual message header may be different. We show the message using rows of 4 bytes. The bytes that are shown using dashes are the one related to the message header.

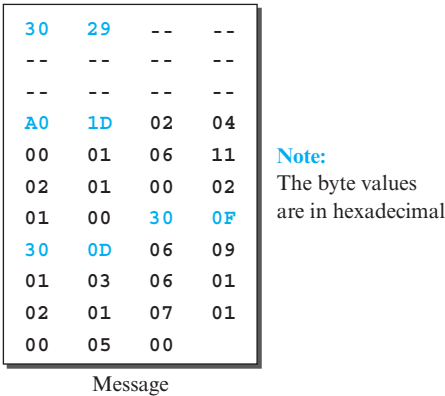
UDP Ports

SNMP uses the services of UDP on two well-known ports, 161 and 162. The well-known port 161 is used by the server (agent), and the well-known port 162 is used by the client (manager).

The agent (server) issues a passive open on port 161. It then waits for a connection from a manager (client). A manager (client) issues an active open using an ephemeral port. The request messages are sent from the client to the server using the ephemeral port as the source port and the well-known port 161 as the destination port. The response messages are sent from the server to the client using the well-known port 161 as the source port and the ephemeral port as the destination port.

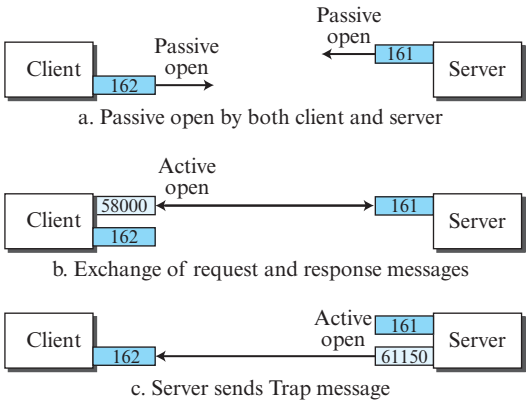
The manager (client) issues a passive open on port 162. It then waits for a connection from an agent (server). Whenever it has a Trap message to send, an agent (server)

Figure 9.21 Actual message sent for Example 9.5



issues an active open, using an ephemeral port. This connection is only one-way, from the server to the client (see Figure 9.22).

Figure 9.22 Port numbers for SNMP



The client-server mechanism in SNMP is different from other protocols. Here both the client and the server use well-known ports. In addition, both the client and the server are running infinitely. The reason is that request messages are initiated by a manager (client), but Trap messages are initiated by an agent (server).

Security

SNMPv3 has added two new features to the previous version: security and remote administration. SNMPv3 allows a manager to choose one or more levels of security

when accessing an agent. Different aspects of security can be configured by the manager to allow message authentication, confidentiality, and integrity.

SNMPv3 also allows remote configuration of security aspects without requiring the administrator to actually be at the place where the device is located.

9.3 ASN.1

In data communication, when we send a continuous stream of bits to a destination, we somehow need to define the format of the data. If we send a *name* and a *number* in a single message, we need to tell the destination that, for example, the first 12 bits define the name and the next 8 bits define the number. We will have more difficulty when we send a complex data type such as an array or a record. For example, in the case of an array, if we send 2000 bits in a message, we need to tell the receiver that it is an array of 200 numbers each of 10 bits or it is an array of 10 numbers each of 200 bits.

A solution is that we separate the definition of data types from the sequence of bits transmitted through the network. This is done through an abstract language that uses some symbols, key words, and atomic data types and lets us make new data types out of the simple types. The language is called Abstract Syntax Notation One (ASN.1). Note that ASN.1 is a very complex language used in different areas of computer science, but in this section, we only introduce the language as much as needed for the SNMP protocol.

9.3.1 Language Basics

Before we show how we can define objects and associated values, let us talk about the language itself. The language use some symbols and some key words and defines some primitive data types. As we said before, SMI uses a subset of these entities in its own language.

Symbols

The language uses a set of symbols, given in Table 9.5. Some of these symbols are single characters, but some are pairs of characters.

Table 9.5 *Symbols used in ASN.1*

<i>Symbol</i>	<i>Meaning</i>	<i>Symbol</i>	<i>Meaning</i>
::=	Defined as or assignment	..	Range
	Or, alternative, or option	{ }	Start and end of a list
—	Negative sign	[]	Start and end of tag
---	The following is a comment	()	Start and end of a subtype

Keywords

The language has a set of limited keywords that can be used. These words can be used in the language only for the purpose for which they have been defined. All words should be in uppercase (Table 9.6).

Table 9.6 Keywords in ASN.1

<i>Keyword</i>	<i>Description</i>
BEGIN	Start of a module
CHOICE	List of alternatives
DEFINITIONS	Definition of a data type or an object
END	End of a module
EXPORTS	Data type that can be exported to other modules
IDENTIFIER	A sequence of non-negative numbers that identifies an object
IMPORTS	Data type defined in an external module and imported
INTEGER	Any positive, zero, or negative integer
NULL	A null value
OBJECT	Used with IDENTIFIER to uniquely define an object
OCTET	Eight-bit binary data
OF	Used with SEQUENCE or SET
SEQUENCE	An ordered list
SEQUENCE OF	An ordered array of data of the same type
SET	An unordered list
SET OF	An array of unordered lists
STRING	A string of data

9.3.2 Data Types

After discussing the symbols and keywords used in the language, it is time to define its data types. The idea is similar to what we see in computer languages such as C, C++, or Java. In ASN.1, we have several simple data types such as integer, float, boolean, char, and so on. We can combine these data types to create a new simple data type (with a different name) or to define a structured data types such as array or struct. We first define simple data types in ASN.1 and then show how to make a new data type of these data types.

Simple Data Types

ASN.1 defines a set of simple (atomic) data types. Each data type is given a universal tag and has a set of values, as shown in Table 9.7. This is the same idea as used in a computer language when we have some basic data types with predefined ranges of values. For example, in C language, we have the data type *int*, which can take a range of values. Note that the tag in the table is actually the rightmost five bits of the tag we defined in Table 9.2.

Table 9.7 Some simple ASN.1 built-in types

<i>Tag</i>	<i>Type</i>	<i>Set of values</i>
Universal 1	BOOLEAN	TRUE or FALSE
Universal 2	INTEGER	Integers (positive, 0, or negative)
Universal 3	BIT STRING	A string of binary digits (bits) or a null set
Universal 4	OCTET STRING	A string of octets or a null set
Universal 5	NULL	Null, single valued
Universal 6	OBJECT IDENTIFIER	A set of values that defines an object
Universal 7	ObjectDescriptor	Human readable text describing an object

Table 9.7 Some simple ASN.1 built-in types (continued)

Tag	Type	Set of values
Universal 8	EXTERNAL	A type which is not in the standard
Universal 9	REAL	Real numbers in scientific notation
Universal 10	ENUMERATED	A list of integers
Universal 16	SEQUENCE, SEQUENCE OF	Ordered list of types
Universal 17	SET, SET OF	Unordered list of types
Universal 18	NumericString	Digits 0-9 and space
Universal 19	PrintableString	printable characters
Universal 26	VisibleString	ISO646String
Universal 27	GeneralString	General character string
Universal 30	CHARACTER STRING	Character set

New Data Types

ASN.1 uses **Backus–Naur Form (BNF)** syntax for defining a new data type from a built-in data type or a previously defined data type as shown below:

<new type> ::= <type>

where the *new type* must start with a capital letter.

Example 9.6

The following is an example of some new types using built-in types from Table 9.7.

```

Married ::= BOOLEAN
MaritalStatus ::= ENUMERATED {single, married, widowed, divorced}
DayOfWeek ::= ENUMERATED {sun, mon, tue, wed, thu, fri, sat}
Age ::= INTEGER

```

New Subtypes

ASN.1 even allows us to create a subtype whose range is a subrange of a built-in type or a previously defined data type.

Example 9.7

The following shows how we can make three new subtypes. The range of the first is the subset of **INTEGER**, the range of the second is the subset of **REAL**, and the range of the third is the subset of **DayOfWeek**, which we defined in Example 9.6. Note that we use the symbol (..) to define the range and the symbol (|) to define the choice.

```

NumberOfStudents ::= INTEGER (15..40)    — An integer with the range 15 to 40
Grade ::= REAL (1.0..5.0)                — A real number with the range 1.0 to 5.0
Weekend ::= DayOfWeek (sun | sat)        — A day that can be sun or sat

```

Simple Variables

In a programming language, we can create a variable of a particular type and assign (store) a value in it. In ASN.1, the term *Value Name* is used instead of variable, but we

use the term *variable*, which is more familiar to programmers. We can create a variable of a particular type and assign a value belonging to the range defined for that type. The following shows the syntax:

<variable> <type> ::= <value>

The name of the variable should start with a lower-case letter to distinguish it from the type.

Example 9.8

The following are a few examples of defining some variables and assigning the appropriate value from the range of those types. Note that the first and the third variables are of the built-in type, the second is of the type defined in Example 9.6, and the last is of a subtype defined in Example 9.7.

```
numberOfComputers INTEGER ::= 2
married Married ::= FALSE
herAge INTEGER ::= 35
classSize NumberOfStudents ::= 22
```

Structured Type

ASN.1 uses a keyword SEQUENCE to define a structured data type similar to *struct* (record) in C language or C++. The SEQUENCE type is an ordered list of variable types. The following shows a new type StudentAccount which is a sequence of three variables: username, password, and accountNumber.

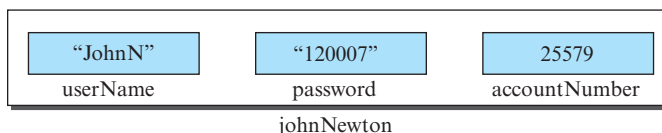
```
StudentAccount ::= SEQUENCE
{
    userName VisibleString,
    password VisibleString,
    accountNumber INTEGER
}
```

Structure Variables

After defining the new type, we can create a variable out of it and assign values to variables as shown below:

```
johnNewton StudentAccount
{
    userName "JohnN",
    password "120007",
    accountNumber 25579
}
```

Figure 9.23 shows the record created from the type definition and value assignments.

Figure 9.23 Record representing the type definition and variable declaration

We use the keyword **SEQUENCE OF** to define a new type similar to an array in C or C++ which is a composite type in which all components are the same. For example, we can define a forwarding table in a router as **SEQUENCE OF Rows** in which each Row is itself a sequence made of several variables.

9.3.3 Encoding

After the data has been defined and values are associated with variables, ASN.1 uses one of the encoding rules to encode the message to be sent. We already discussed the Basic Encoding Rule in the previous section.

9.4 END-CHAPTER MATERIALS

9.4.1 Further Reading

For more details about subjects discussed in this chapter, we recommend the following books and RFCs. The items enclosed in brackets refer to the reference list at the end of the book.

Books

Several books give thorough coverage of SNMP: [Com 06], [Ste 94], [Tan 03], and [MS 01]

RFCs

Several RFCs show different updates on SMTP, including RFC 3410, RFC 3412, RFC 3415, and RFC 3418. More information about MIB can be found in RFC 2578, RFC 2579, and RFC 2580.

9.4.2 Key Terms

Abstract Syntax Notation One (ASN.1)	Simple Network Management Protocol (SNMP)
Backus–Naur Form (BNF)	Structure of Management Information (SMI)
Basic Encoding Rules (BER)	structured data type
Management Information Base (MIB)	Trap
object identifier	
simple data type	

9.4.3 Summary

The five areas comprising network management are configuration management, fault management, performance management, accounting management, and security management. Configuration management is concerned with the physical or logical

changes of network entities. Fault management is concerned with the proper operation of each network component. Performance management is concerned with the monitoring and controlling of the network to ensure the network runs as efficiently as possible. Security management is concerned with controlling access to the network. Accounting management is concerned with controlling user access to network resources through charges.

Simple Network Management Protocol (SNMP) is a framework for managing devices in an internet using the TCP/IP protocol suite. A manager, usually a host, controls and monitors a set of agents, usually routers. SNMP uses the services of SMI and MIB. SMI names objects, defines the type of data that can be stored in an object, and encodes the data. MIB is a collection of groups of objects that can be managed by SNMP. MIB uses lexicographic ordering to manage its variables.

Abstract Syntax Notation Number One (ASN.1) is a language that defines the syntax and semantics of data. It uses some symbols, keywords, simple and structured data types. Part of ASN.1 is used by SMI to define the format of objects and values used in network management.

9.5 PRACTICE SET

9.5.1 Quizzes

A set of interactive quizzes for this chapter can be found on the book website. It is strongly recommended that students take the quizzes to check his/her understanding of the materials before continuing with the practice set.

9.5.2 Questions

- Q9-1.** A network manager decides to replace the old router that connects the organization to the Internet with a more powerful one. What area of network management is involved here?
- Q9-2.** A network manager decides to replace a version of accounting software with a new version. What area of network management is involved here?
- Q9-3.** Distinguish between reactive fault management and proactive fault management.
- Q9-4.** If network management does not replace a component whose lifetime has been expired, what area in network management has been ignored?
- Q9-5.** Which of the following is not one of the five areas of network management defined by ISO?
- a. fault b. performance c. personnel
- Q9-6.** Which of the following is not part of configuration management?
- a. reconfiguration b. encryption c. documentation
- Q9-7.** Which of the following devices cannot be a manager station in SNMP?
- a. a router b. a host c. a switch
- Q9-8.** Does an SNMP manager run a client SNMP program or a server SNMP program?

- Q9-9.** Distinguish between internal and external data traffic in an organization.
- Q9-10.** If a student in a college can monopolize access to a piece of software, causing other students to wait for a long time, which area of network management has failed?
- Q9-11.** Find the type (simple, sequence, sequence of) of the following objects in SMI.
- An unsigned integer
 - An IP address
 - An object name
 - A list of integers
 - A record defining an object name, an IP address, and an integer
 - A list of records in which each record is an object name followed by a counter.
- Q9-12.** What is the length of the value field in the following BER encoding?
- ```
04 09 48 65 6C 4C ...
```
- Q9-13.** Show how the textual name “iso.org.dod” is numerically encoded in SMI.
- Q9-14.** Is it possible to have a textual name in SMI as “iso.org.internet”? Explain.
- Q9-15.** Can an SNMP message reference a leaf node in the MIB tree? Explain.
- Q9-16.** Assume that a manageable object has only three simple variables. How many leaves can be found in the MIB tree for this object?
- Q9-17.** Distinguish between SMI and MIB.
- Q9-18.** What does the *if* object in MIB define? Why does this object need to be managed?
- Q9-19.** Assume that an object identifier in MIB has three simple variables. If the object identifier is  $x$ , what is the identifier of each variable?
- Q9-20.** Can SNMP reference the entire row of a table? In other words, can SNMP retrieve or change the values in the entire row of a table? Explain.
- Q9-21.** In SNMP, which of the following PDUs are sent from a client SNMP to a server SNMP?
- GetRequest
  - Response
  - Trap
- Q9-22.** What are the source and destination port numbers when an SNMP message carries one of the following PDUs?
- GetRequest
  - Response
  - Trap
  - Report
- Q9-23.** Assume that a manageable object has a table with three columns. How many leaves are there in the MIB tree for this table?
- Q9-24.** Distinguish between a GetRequest PDU and a SetRequest PDU.

### 9.5.3 Problems

- P9-1.** Assume that an object  $x$  has two simple variables and one table with two columns. What is the identifier for each variable and each column of the table? We assume that simple variables come before the table.

- P9-2.** Assume that an object  $x$  has one simple variable and two tables with two and three columns, respectively. What is the identifier for the variable and each column of each table? We assume that the simple variable comes before the tables.
- P9-3.** Assume that an object  $x$  has two simple variables: an integer and an IP address. What is the identifier for each variable?
- P9-4.** Assume that an object  $x$  has only one table with two columns. What is the identifier for each column?
- P9-5.** One of the objects (groups) that can be managed is the *ip* group with the object identifier (1.3.6.1.2.1.4) in which (1.3.6.1.2.1) is the identifier of MIB-2 and (4) defines the *ip* group. In an agent, this object has 20 simple variables and three tables. One of the tables is the routing (forwarding) table with the identifier (1.3.6.1.2.1.4.21). This table has eleven columns, the first of which is called the *ipRouteDes*, which means the destination IP address. Assume that the indexing is based on the first column. Assume that the table has four rows at the moment with the destination IP addresses (201.14.67.0), (123.16.0.0), (11.0.0.0), and (0.0.0.0). Show how SNMP can access all four instances of the second column, called *ipRouteIfIndex*, which defines the interface numbers through which the IP should be sent out.
- P9-6.** Show the encoding for the INTEGER 1456 using BER.
- P9-7.** Show the encoding for the OCTET STRING “Hello world.” using BER.
- P9-8.** Show the encoding for the IPAddress 112.56.23.78 using BER.
- P9-9.** Object  $x$  has two simple variables. How can SNMP refer to the instance of each variable?
- P9-10.** Object  $x$  has one table with two columns. The table at this moment has three rows with the contents shown below. If the table index is based on the values in the first column, show how SNMP can access each instance.

**Object  $x$**

|   |    |
|---|----|
| a | aa |
| b | bb |
| c | cc |

**Table**

- P9-11.** Assume that a data structure is made of an INTEGER of value (131) and another structure made of an IPAddress of value (24.70.6.14) and an OCTET-STRING (“UDP”). Using BER, encode the data structure.
- P9-12.** Given the code 02040000C738, decode it using BER.
- P9-13.** Given the code 300C02040000099806040A05030E, decode it using BER.
- P9-14.** Given the code 300D04024E6F300706030103060500, decode it using BER.
- P9-15.** Show the encoding for the object identifier 1.3.6.1.2.1.7.1 (the udpInData-gram variable in udp group) using BER.
- P9-16.** Using BER, show how we can encode a structured data type made of an INTEGER of value (2371), an OCTET STRING of value (“Computer”), and an IPAddress of value (185.32.1.5) as shown below:

```

SEQUENCE
{
 INTEGER 2371
 OCTET STRING "Computer"
 IP Address 185.32.1.5
}

```

- P9-17.** Define a GetRequest PDU (*see* Figure 9.18) using the syntax defined for structured data types in ASN.1.
- P9-18.** Define a Response PDU (*see* Figure 9.18) using the syntax defined for structured data types in ASN.1.
- P9-19.** Assume that a manager needs to know the number of user datagrams an agent has sent out (a udpOutDatagrams counter with the identifier 1.3.6.1.2.1.7.4). Show the code for a Varbind that is sent in a GetRequest message and the code that the agent will send in the Response message if the value of the counter at this moment is 15.
- P9-20.** Define an SNMP message (*see* Figure 9.19) using the syntax defined for structured data types in ASN.1.
- P9-21.** Define a VarbindList (*see* Figure 9.19) using the syntax defined for structured data types in ASN.1.